

Sistemas Distribuídos

Engenharia de Telecomunicações

Prof. Emerson Ribeiro de Mello

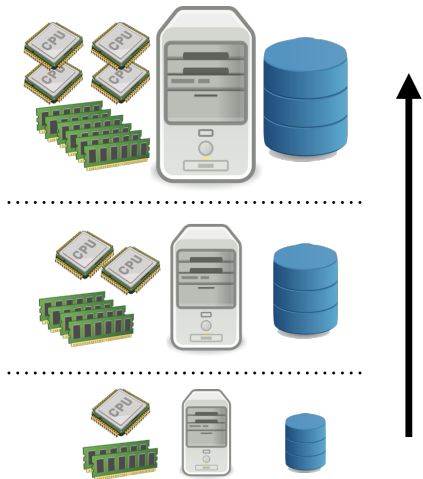
mello@ifsc.edu.br

Licenciamento



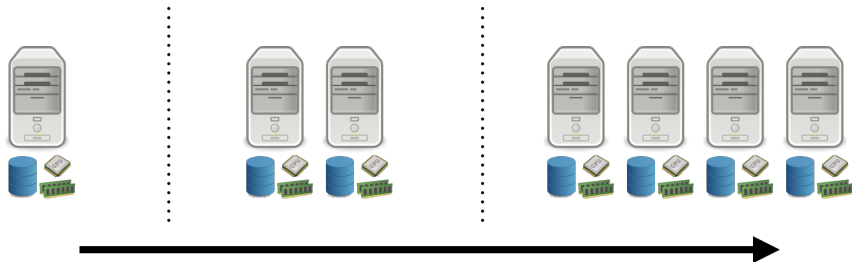
Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Escalonamento vertical



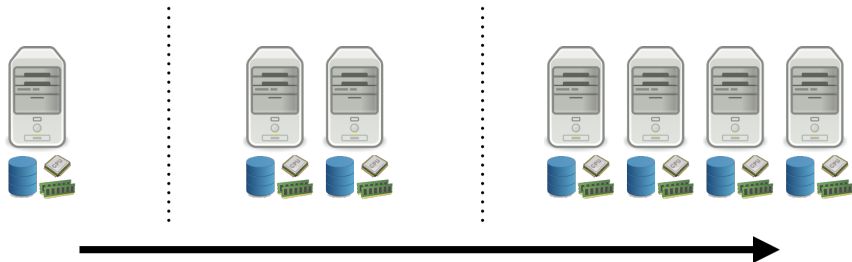
Adiciona mais recursos (memória, cpu, disco) em um único nó para permitir que atenda o aumento da demanda

Escalonamento horizontal



- Capacidade de processamento e armazenamento é aumentada por meio de adição de novos nós no *cluster*

Escalonamento horizontal



- Capacidade de processamento e armazenamento é aumentada por meio de adição de novos nós no *cluster*

Escalabilidade horizontal é a opção mais barata e flexível, porém possui uma maior complexidade

- Sistema centralizado vs sistema distribuído

Servidor de 1U para rack



Fonte: HP

Servidor BladeSystem de 10U



Fonte: HP

Storage



Por que Sistemas Distribuídos?

Revisão da 1a. aula

■ Aplicações naturalmente distribuídas

- Exemplos: sistemas acadêmicos, navegação web, monitoramento remoto

■ Desempenho e escalabilidade

- Distribuição de tarefas entre vários computadores
- Facilidade para expandir capacidade de processamento
- Melhor relação desempenho/custo

■ Operação em larga escala

- Suporte a usuários e recursos dispersos geograficamente

■ Alta disponibilidade

- Funcionamento contínuo mesmo diante de falhas parciais

Introdução aos Sistemas Distribuídos I

Revisão da 1a. aula

Principais desafios em sistemas distribuídos

- Compartilhamento de dados e processamento entre múltiplos computadores
- Descoberta de serviços: localizar e acessar recursos distribuídos
- Complexidade de desenvolvimento: sistemas maiores e comportamento menos previsível

Introdução aos Sistemas Distribuídos II

Revisão da 1a. aula

Gestão de falhas

■ Sistemas centralizados:

- Falha total: todo o sistema para de funcionar
- Usuário percebe imediatamente a falha
- Recuperação geralmente exige reiniciar o sistema

■ Sistemas distribuídos:

- Falhas podem ser parciais: apenas parte do sistema é afetada
- Usuário pode não perceber a falha imediatamente
- Estratégias de tolerância a falhas permitem continuidade do serviço

Introdução aos Sistemas Distribuídos

Principais objetivos de um SD

- 1 **Acessibilidade**
- 2 **Transparência**
- 3 **Abertura**
- 4 **Escalabilidade**
- 5 **Concorrência**

Acessibilidade de Recursos

Permitir que usuários (pessoas ou aplicações) acessem recursos de forma **simples, eficiente e segura**

- Exemplos: impressoras, computadores, armazenamento, arquivos, etc.
- O **compartilhamento de recursos** aumenta a colaboração, mas exige atenção à **segurança**.
 - Riscos: acesso não autorizado, uso indevido, privacidade dos usuários

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- Localização
- Desempenho
- Mobilidade
- Replicação
- Concorrência
- Falhas

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- Localização
- Desempenho
- Mobilidade
- Replicação
- Concorrência
- Falhas

Esconde diferenças de representação e manipulação dos dados em diferentes sistemas.

- Exemplo: Convenções de nomes de arquivos e formatos em sistemas operacionais distintos.

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- **Localização**
- Desempenho
- Mobilidade
- Replicação
- Concorrência
- Falhas

Esconde onde os recursos estão fisicamente.

- Exemplo: Ao acessar <https://emersonmello.me>, não é possível saber onde o servidor está localizado.

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- Localização
- **Desempenho**
- Mobilidade
- Replicação
- Concorrência
- Falhas

Permite reconfigurar o sistema para melhorar a performance conforme a demanda.

- Exemplo: Adição de servidores web para balancear carga em períodos críticos.

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- Localização
 - Esconde mudanças de localização dos recursos durante o acesso.
- Desempenho
- Mobilidade
 - Exemplo: Chamada VoIP no celular enquanto se movimenta pelo campus sem interrupção.
- Replicação
- Concorrência
- Falhas

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- Localização
- Desempenho
- Mobilidade
- **Replicação**
- Concorrência
- Falhas

Esconde que recursos são replicados para aumentar disponibilidade ou desempenho.

- Exemplo: Ao acessar o Google, o usuário não sabe em qual servidor está conectado.

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
 - Localização
 - Desempenho
 - Mobilidade
 - Replicação
 - Concorrência
 - Falhas
- Esconde que vários usuários podem acessar o mesmo recurso simultaneamente.
- Exemplo: Dois usuários acessando a mesma tabela no banco de dados; o sistema garante consistência.

Transparência

Transparência significa **esconder dos usuários** que o processamento e os recursos estão distribuídos em vários computadores

Tipos

- Acesso
- Localização
- Desempenho
- Mobilidade
- Replicação
- Concorrência
- Falhas

Esconde falhas e recuperações dos recursos.

- Exemplo: Se uma réplica de servidor web falhar, o serviço permanece disponível e o usuário não percebe a recuperação.

Transparência

- Um sistema distribuído **nunca será totalmente transparente**
 - A latência da comunicação pela Internet não pode ser ocultada do usuário
 - Falhas podem ocorrer e nem sempre podem ser mascaradas pelo sistema
- **Em alguns casos, é importante que o usuário perceba a distribuição**
 - Por exemplo: o usuário precisa saber onde está a impressora para retirar o documento

Abertura em Sistemas Distribuídos I

O que significa ser aberto?

Um sistema aberto pode ser facilmente estendido, adaptado ou integrado a outros sistemas, permitindo diferentes implementações.

■ Exemplos:

- **Computador pessoal:** Permite conectar periféricos de diferentes fabricantes.
- **Rede TCP/IP:** Dispositivos com arquiteturas e sistemas operacionais distintos comunicam-se sem dificuldades.
- Protocolos e interfaces abertos facilitam a interoperabilidade. Exemplo: especificações públicas em www.ietf.org

Sistema Distribuído Aberto

- Um sistema distribuído aberto é formado por **componentes modulares**, que podem ser **facilmente adaptados ou substituídos**.
 - Interfaces públicas e internas devem ser bem definidas e documentadas para facilitar integração e evolução do sistema.
- Separar **políticas** de **mecanismos** aumenta a flexibilidade:
 - **Mecanismo**: cache do navegador web
 - **Política**: usuário decide o que salvar e por quanto tempo

Sistema Distribuído Aberto

Curiosidade em sistemas operacionais

Sistemas monolíticos x sistemas abertos

- Sistemas monolíticos tendem a ser fechados e difíceis de modificar.
- Exemplo: Minix utiliza um microkernel, onde apenas o núcleo roda em modo kernel e o restante do sistema opera como módulos isolados em modo usuário, facilitando adaptação e evolução.
- Em contraste, o Linux possui um kernel monolítico, menos flexível para mudanças.
- Projetar sistemas totalmente abertos é desafiador e exige interfaces bem definidas e modularidade.

Escalabilidade

- **Sistemas centralizados têm baixa escalabilidade**

- Exemplo: Um único servidor de arquivos centralizado

- Com o aumento de usuários e recursos, é necessário ampliar processamento, armazenamento e banda de rede
 - Usuários distantes aumentam a latência de acesso ao servidor

Escalabilidade

- **Sistemas centralizados têm baixa escalabilidade**

- Exemplo: Um único servidor de arquivos centralizado

- Com o aumento de usuários e recursos, é necessário ampliar processamento, armazenamento e banda de rede

- Usuários distantes aumentam a latência de acesso ao servidor

- Um **sistema escalável** mantém desempenho e funcionalidade mesmo com o crescimento de usuários e recursos

- Escalabilidade pode ser analisada em três dimensões:

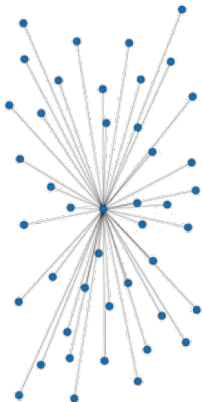
- Tamanho: quantidade de usuários e recursos

- Dispersão geográfica: distância entre os nós

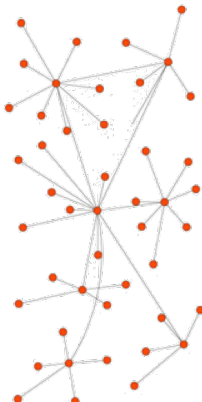
- Administração: número de domínios administrativos

Escalabilidade

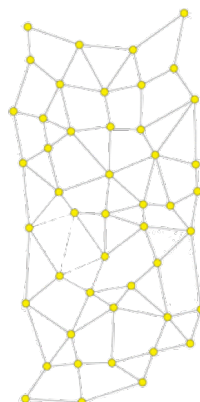
Qual topologia é mais escalável?



Centralizada



Descentralizada



Distribuída

Desafios para projetar um sistema distribuído escalável

■ Custo dos recursos físicos

- O sistema deve permitir expansão com investimento razoável para atender aumento de demanda.

■ Desempenho

- Estruturas hierárquicas (ex: DNS) são escaláveis, mas podem reduzir o desempenho em relação a estruturas planas.
- Comunicação síncrona funciona bem em redes locais, mas redes WAN exigem comunicação assíncrona.

■ Esgotamento de recursos de software

- Recursos como endereços IPv4 podem acabar; é difícil prever o crescimento futuro.

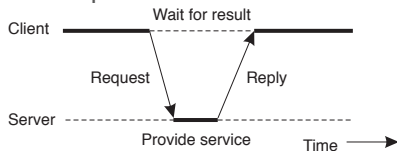
■ Gargalos

- Algoritmos descentralizados (ex: DNS) ajudam a evitar pontos únicos de falha ou sobrecarga.

Técnicas para projetar sistema distribuído escalável

Ocultar a latência inerente da comunicação

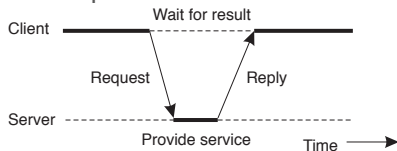
- **Comunicação síncrona** – o cliente só prossegue em sua execução após receber a resposta de seu pedido enviado ao servidor



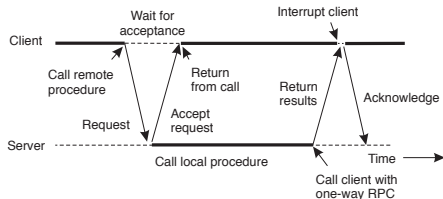
Técnicas para projetar sistema distribuído escalável

Ocultar a latência inerente da comunicação

- **Comunicação síncrona** – o cliente só prossegue em sua execução após receber a resposta de seu pedido enviado ao servidor



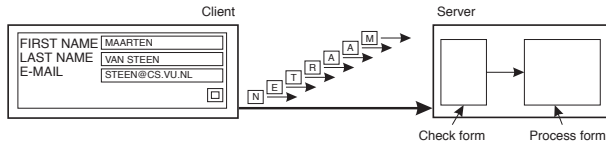
- **Comunicação assíncrona** – evita bloquear o cliente enquanto aguarda pela resposta do servidor



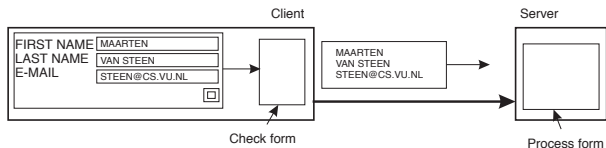
Técnicas para projetar sistema distribuído escalável

Ocultar a latência inerente da comunicação

- **Comunicação assíncrona não é adequada** para **aplicações interativas**, onde o usuário aguarda por uma resposta rápida
- **Mover para o cliente** a maior parte possível de **tarefas de processamento**, reduzirá carga no servidor



(a)



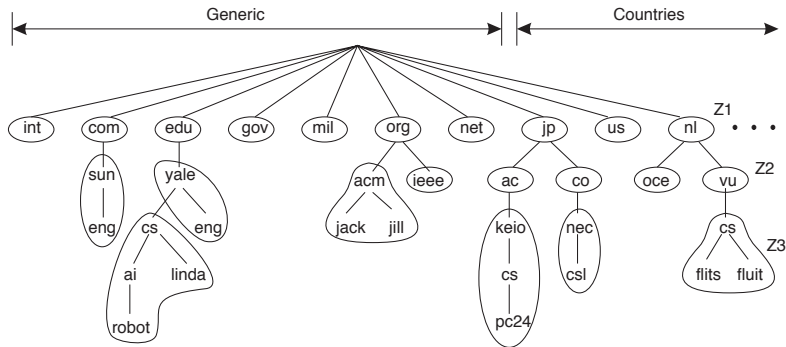
(b)

É preferível **enviar uma única mensagem grande**, do que várias mensagens pequenas

Técnicas para projetar sistema distribuído escalável

Distribuição das responsabilidades

- Divida um componente em diversas pequenas partes e as espalhe pelo sistema
- Cada parte fica com uma pequena responsabilidade de todo o sistema



Técnicas para projetar sistema distribuído escalável

Replicação dos componentes

- Replicação aumenta a disponibilidade e distribui a carga entre servidores
 - Réplicas próximas dos clientes reduzem a latência e melhoram o desempenho
- O uso de **cache** em servidores DNS é um exemplo de replicação
 - Na primeira consulta, o servidor busca a resposta e armazena no cache
 - Consultas seguintes são atendidas diretamente do cache
- O maior desafio é manter a consistência dos dados entre as réplicas
 - Exemplo: tempo de validade dos registros no cache do DNS

Concorrência

- Recursos distribuídos podem ser acessados simultaneamente por vários clientes
 - É comum múltiplos clientes tentarem acessar o mesmo recurso ao mesmo tempo
- É necessário garantir a consistência dos dados entre réplicas e acessos concorrentes
 - Operações devem ser sincronizadas e ordenadas
 - O conceito de semáforo em sistemas operacionais é utilizado para controlar o acesso concorrente

Resumo: Sistemas Distribuídos

- **Compartilhamento de recursos** é o principal motivo para adotar sistemas distribuídos
- **Transparência** esconde dos usuários a distribuição dos recursos e do processamento
- **Modularidade** e uso de **padrões abertos** facilitam integração, evolução e interoperabilidade
- **Distribuição de responsabilidades** e **replicação** aumentam escalabilidade e disponibilidade
- É necessário garantir **consistência** dos dados mesmo com acessos concorrentes

Resumo: Desafios e Considerações

- Redes locais e redes de longa distância apresentam diferenças importantes
 - Segurança, confiabilidade, latência, heterogeneidade, taxa de transferência e administração
- Sistemas distribuídos exigem soluções específicas para lidar com falhas, escalabilidade e integração

Aulas baseadas em



TANENBAUM, Andrew S.; STEEN, Maarten van
Sistemas Distribuidos: Princípios e paradigmas

- Veja Capítulo 1 do livro *Distributed Systems* 3rd edition